SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA0609	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	**Total Marks:**	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Yuvasri.B	**Reg. No.:**	192421231
Section / Batch:	Slot B / BTECH-IT	**Date:**	06/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Case Study Report
Event Log Monitoring System Using Sorting and Binary Search

Description of the Real-World Problem

Modern organizations such as banks, hospitals, e-commerce companies, cloud platforms, and cybersecurity firms generate millions of event logs every day. An event log records system activities such as user logins, file access, software updates, network requests, security alerts, and application errors.

As the amount of data increases, searching for a specific event becomes difficult and time-consuming. For example, if a security analyst wants to find an event with Event ID **2045** among one million records, checking each record one by one is inefficient.

To improve efficiency, event logs are first arranged in sorted order using a sorting algorithm. Once sorted, Binary Search is used to locate the required event quickly.

This combination helps organizations monitor systems efficiently while reducing search time and improving overall performance.

Problem Statement

Design an Event Log Monitoring System that:

- Stores multiple event IDs.
- Sorts the event IDs in ascending order.
- Searches for a required Event ID using Binary Search.
- Displays whether the event exists and its position.
- Provides fast searching even when the number of logs is very large.

Algorithm Used

1. Merge Sort (Divide and Conquer)

Merge Sort divides the array into smaller halves until each subarray contains one element. Then it merges the subarrays in sorted order.

Steps

1. Divide the array into two halves.
2. Continue dividing until only one element remains.
3. Merge two sorted arrays.
4. Continue merging until the entire array becomes sorted.

2. Binary Search

Binary Search works only on sorted data.

Steps

1. Find the middle element.
2. If the key equals the middle element, return its position.
3. If the key is smaller, search the left half.
4. Otherwise search the right half.
5. Continue until the element is found or the search space becomes empty.

Justification for Choosing the Algorithm

Why Merge Sort?

- Works efficiently on very large datasets.
- Guarantees O(n log n) time complexity.
- Stable sorting algorithm.
- Uses Divide and Conquer strategy.
- Suitable for external storage and large log files.

Why Binary Search?

- Very fast searching algorithm.
- Requires only O(log n) comparisons.
- Ideal for sorted event logs.
- Reduces response time significantly.

Why Combine Both?

Sorting once allows multiple searches to be performed efficiently.
Instead of repeatedly using Linear Search,
Sort Once → Search Many Times
This approach greatly improves performance in monitoring systems.

Complexity Analysis

Merge Sort

Operation	**Complexity**
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	O(n log n)
Extra Space	O(n)

Binary Search

Operation	**Complexity**
Best Case	O(1)
Average Case	O(log n)
Worst Case	O(log n)
Space	O(1)

Overall Complexity

Sorting:

O(n log n)

Searching:
O(log n)
Total:
O(n log n) + O(log n)
Since sorting is performed only once, future searches remain extremely fast.

Manual Execution (Step-by-Step Process)

Suppose the event IDs are
45 12 78 34 90 21 56
Step 1
Divide
45 12 78 | 34 90 21 56
Step 2
Further Divide
45 | 12 | 78 | 34 | 90 | 21 | 56

Step 3
Merge
12 45
34 90
21 56

Step 4
Merge Again
12 45 78
21 34 56 90

Step 5
Final Sorted List
12 21 34 45 56 78 90

Suppose the user searches
56

Binary Search

Low	Mid	High	Middle Value
0	3	6	45

56 > 45

Search Right Half

Low	Mid	High	Middle Value
4	4	4	56

Applications

Sorting and Binary Search are widely used in:

- Network Monitoring Systems
- Cloud Computing Platforms
- Database Indexing
- Banking Transaction Logs
- Hospital Information Systems
- Security Information and Event Management (SIEM)
- Operating System Event Logs
- Web Server Monitoring
- E-commerce Transaction Logs
- Cybersecurity Incident Detection

These applications require fast searching among millions of records.

Advantages

Merge Sort

- Stable sorting algorithm.
- Predictable performance.
- Efficient for large datasets.
- Uses Divide and Conquer.
- Performs well on linked lists.

Binary Search

- Extremely fast searching.
- Reduces execution time.
- Suitable for repeated searches.
- Requires very few comparisons.
- Ideal for sorted databases.

Limitations

Merge Sort

- Requires additional memory.
- Slightly slower than Quick Sort on small datasets.
- Extra copying during merging.

Binary Search

- Cannot be applied on unsorted data.

- Sorting is mandatory.
- Not efficient for frequently changing datasets.

Comparison with Other Algorithms

Algorithm	Best	Average	Worst	Remarks
Linear Search	O(1)	O(n)	O(n)	Simple but slow
Binary Search	O(1)	O(log n)	O(log n)	Fastest for sorted data
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	Very slow
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Simple
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	Good for small data
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	Very fast average case
Merge Sort	O(n log n)	O(n log n)	O(n log n)	Stable and reliable

Python Program:

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
```

```
            arr[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid

        elif arr[mid] < key:
            low = mid + 1

        else:
            high = mid - 1

    return -1

arr = [45, 12, 78, 34, 90, 21, 56]

merge_sort(arr)

print("Sorted Event Logs:", arr)

key = int(input("Enter Event ID to search: "))

result = binary_search(arr, key)

if result != -1:
    print("Event Found at Position:", result + 1)
else:
    print("Event Not Found")
```

csae study.py - C:/Users/yuvas/Downloads/win-g2010-1_3-n_mcd/win/LIB/DAA/Assessment/csae study.py (3.14.6)

File Edit Format Run Options Window Help

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid

        elif arr[mid] < key:
            low = mid + 1

        else:
            high = mid - 1

    return -1
```

Output:

IDLE Shell 3.14.6

File Edit Shell Debug Options Window Help

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/yuvas/Downloads/win-g2010-1_3-n_mcd/win/LIB/DAA/Assessment/csae study.py
Sorted Event Logs: [12, 21, 34, 45, 56, 78, 90]
Enter Event ID to search: 56
Event Found at Position: 5
>>>
```

Conclusion

The Event Log Monitoring System demonstrates how sorting and searching algorithms work together to improve system efficiency. Merge Sort organizes event logs using the Divide and Conquer strategy, ensuring predictable O(n log n) performance even for large datasets. Once the logs are sorted, Binary Search enables rapid retrieval of events with O(log n) time complexity.

This combination significantly reduces response time, making it suitable for real-world applications such as cybersecurity, cloud computing, banking, healthcare, and database management. Although Merge Sort requires additional memory and Binary Search depends on sorted data, their combined advantages outweigh these limitations in systems where frequent searches are required.

Hence, Merge Sort and Binary Search provide an efficient, scalable, and reliable solution for modern Event Log Monitoring Systems.

Code of Conduct

I (Yuvasri /192421231) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:Yuvasri B

RUBRICS FOR CALCULATION

Case Study					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts

Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____